

Usage

Whilst this library extrapolates the need for understanding the specific **DCC-EX** native commands from a throttle developer, it is highly recommended to familiarise yourself with the concepts outlined in the <https://dcc-ex.com/throttles/tech-reference.html>.

Setup

Once the *DCCExProtocol* object is instantiated, a connection must be made to the **EX-CommandStation** using the *connect(&stream)* method and providing a suitable Arduino Stream, such as a WiFi client or serial connection.

It is also recommended to enable logging to an Arduino Stream using the *setLogStream(&stream)* method.

For WiFi clients, long periods of no interactive commands being sent may cause the WiFi client to be disconnected, so it is recommended to enable heartbeats for these, which defaults to sending a heartbeat every 60 seconds. If commands are sent regularly, no heartbeats are sent.

An example using an ESP32 with WiFi to connect to EX-CommandStation, with logging to the serial console:

```
WiFiClient client;
DCCExProtocol dccexProtocol;

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  while(WiFi.status() != WL_CONNECTED) delay(1000);
  if (!client.connect(serverAddress, serverPort)) {
    while(1) delay(1000);
  }
  dccexProtocol.setLogStream(&Serial);
  dccexProtocol.enableHeartbeat();
  dccexProtocol.connect(&client);
}

void loop() {
  dccexProtocol.check();
  // other code here
}
```

On this page

- [Setup](#)
 - [A Note on DCCExProtocolDelegate](#)
- [Control and Inputs](#)
- [Retrieving and referring to object lists](#)
 - [Command Station Consist \(CSConsist\) list](#)
- [Speed Update Queuing and Throttling](#)
- [Object ownership](#)



An example using STM32F103C8 Bluepill with hardware serial port 1 connecting to EX-CommandStation, and logging to the USB serial console:

```
DCCEXProtocol dccexProtocol;

void setup() {
  Serial.begin(115200);
  Serial1.begin(115200);
  dccexProtocol.setLogStream(&Serial);
  dccexProtocol.connect(&Serial1);
}

void loop() {
  dccexProtocol.check();
  // other code here
}
```

⚠ Warning

When using hardware serial interfaces, be aware of the default Rx and Tx buffer sizes. When using the STM32 Bluepill in particular, the default buffer size is 64KB for each. This means only small lists (roster, turnouts, etc.) can successfully be retrieved before the buffer fills, especially when updating other physical devices such as displays needs to be performed within the main loop.

The resolution to this issue is to increase the buffer size, which can be accomplished by adding the appropriate build flags to PlatformIO:

```
build_flags =
    -DSERIAL_RX_BUFFER_SIZE=256
    -DSERIAL_TX_BUFFER_SIZE=256
```

As covered in the design principles above, you must include the *check()* method as often as possible to receive command responses and broadcasts and have these processed by the library and any event handlers defined in your custom *DCCEXProtocolDelegate* class.

Refer to the [Examples](#) to see how this may be implemented.

A Note on *DCCEXProtocolDelegate*

New in DCCEXProtocol 1.3.0

In previous versions of the library, it was necessary to create a *DCCEXProtocolDelegate* instance and associate it with your *DCCEXProtocol* instance to be able to send any commands to a command station, even if there was no reliance on any *DCCEXProtocolDelegate* methods.

This has now been corrected, and the library can be utilised without a `DCCEXProtocolDelegate` instance. The `dccexProtocol.check()` method is responsible for managing and maintaining the state of the various objects, and the `DCCEXProtocolDelegate` methods simply notify when broadcasts or responses are received.

However, in case it's not obvious, if your software needs to respond to a response or broadcast (eg. changing a display when a turnout is thrown), then you should still utilise the `DCCEXProtocolDelegate` to be notified when these events occur, rather than having to manually poll any objects for changes.

Control and Inputs

It is up to the client software utilising this library to manage control and input methods to provide input into the protocol functions such as setting locomotive speed and direction, turning functions on and off, and controlling the various other objects and methods available.

For example, multiple rotary encoders may be used to simultaneously control multiple locomotives or consists.

There is, however, no need to instantiate more than one `DCCEXProtocol` or `DCCEXProtocolDelegate` object providing the client software is written appropriately, and we recommend creating a custom class that can take the `DCCEXProtocol` object as a parameter to enable this.

See the `DCCEXProtocol_Multi_Throttle_Control` example for an idea of how this may be implemented.

A further note is that controls and inputs should be passed to the protocol only, and should not update local references to object attributes (such as speed and direction), but rather that the responses to these inputs as received by the protocol and delegate events should be used to update local references.

In this manner, the user of the throttle/client software will see the true results of their inputs which will reflect what EX-CommandStation is doing in response to those inputs.

Retrieving and referring to object lists

To retrieve the various objects lists from **EX-CommandStation**, use the `getLists(bool rosterRequired, bool turnoutListRequired, bool routeListRequired, bool turntableListRequired)` method within your `loop()` function to ensure these are retrieved successfully.

As of version 1.2.1 of this library, the method defaults to all lists true, meaning you can simply call it without parameters.

```
// Explicitly retrieve all lists
dccexProtocol.getLists(true, true, true, true);
// Default retrieves all lists as of 1.2.1
dccexProtocol.getLists();
// Only retrieve the roster
dccexProtocol.getLists(true, false, false, false);
```

If you have a lot of defined objects in your **EX-CommandStation** (eg. 50+ turnouts or 50+ roster entries), you will likely need to increase the maximum number of parameters allowed when defining the DCCExProtocol instance which is now a configurable parameter as of version 1.0.0 of the library.

You can set the command buffer size and parameter count:

```
// dccexProtocol(maxCmdBuffer, maxCommandParams);
DCCExProtocol dccexProtocol; // Use default 500 byte buffer, 50 parameters
DCCExProtocol dccexProtocol(500, 100); // Use default 500 byte buffer, 100 parameters
```

All objects are contained within linked lists and can be accessed via for loops:

```
for (Loco* loco=dccexProtocol.roster->getFirst(); loco; loco=loco->getNext()) {
    // loco methods are available here
}

for (Turnout* turnout=dccexProtocol.turnouts->getFirst(); turnout; turnout=turnout->getNext()) {
    // turnout methods are available here
}

for (Route* route=dccexProtocol.routes->getFirst(); route; route=route->getNext()) {
    // route methods are available here
}

for (Turntable* turntable=dccexProtocol.turntables->getFirst(); turntable;
turntable=turntable->getNext()) {
    // turntable methods are available here
    for (TurntableIndex* ttIndex=turntable->getFirstIndex(); ttIndex; ttIndex=ttIndex->getNextIndex()) {
        // turntable index methods are available here
    }
}
```

Refer to the `DCCExProtocol_Roster_etc` example for an idea of how this may be implemented.

Command Station Consist (CSConsist) list

New in DCCExProtocol 1.3.0

With the introduction of the Command Station Consist (CSConsist) in EX-CommandStation, there is now also a list of CSConsists available. However, this list is dynamic and is updated each time a CSConsist is created, modified, or deleted, and therefore should not be treated the same as the roster, turnout, route, or turntable lists.

Note that this new CSConsist support deprecates the existing Consist object and all associated methods, and these will be removed in a future release.

To get the current list of CSConsists use this method:

```
dccexProtocol.requestCSConsists();
```

The command station will respond and create a list of CSConsist objects on the next call to `dccexProtocol.check()`.

Similar to the lists above, this can be navigated like so:

```
for (CSConsist *csConsist = dccexProtocol.csConsists->getFirst(); csConsist; csConsist = csConsist->getNext()) {  
    // CSConsist methods are available here  
}
```

Refer to the library method documentation for the various methods available to create, update, and delete these new CSConsists.

Speed Update Queuing and Throttling

New in DCCExProtocol 1.3.0

To prevent excessive throttle speed adjustments from saturating WiFi/serial buffers and causing excessive broadcasts to other throttles, buffering of loco speed and direction (`setThrottle(...)`) commands are now throttled using a timed queue. By default, this timer is set at 100 milliseconds, which is typically not noticeable by a throttle user.

If you wish to adjust this interval, it can be done so when creating a DCCExProtocol instance by passing an additional parameter to the command buffer and parameters outlined above:

```
// dccexProtocol(maxCmdBuffer, maxCommandParams, userChangeDelay);  
DCCExProtocol dccexProtocol; // Use default 500 byte buffer, 50 parameters, and 100ms user  
change delay  
DCCExProtocol dccexProtocol(500, 100, 50); // Use default 500 byte buffer, 100 parameters,  
and decrease delay to 50ms
```

No other commands are processed through this mechanism, meaning every other command is still sent instantly.

Object ownership

All objects created by the *getLists()* method are owned by the *DCCEXProtocol* instance, so if, for example, you wish to “forget” a Loco and disassociate from some form of throttle software, **do not delete** the Loco object as this will break the roster linked list.

Each object type has a clear method to delete the list of retrieved objects:

- Loco: *clearRoster()*
- Route: *clearRouteList()*
- Turnout: *clearTurnoutList()*
- Turntable: *clearTurntableList()*

You can also clear all lists with *dccexProtocol.clearAllLists()*.

Note there is one exception to this rule which is Loco objects that are created with the *LocoSource::LocoSourceEntry* type set, as these do not get added to the roster list.

Updated in DCCEXProtocol 1.3.0

If you create a Loco object using this type, you may delete the object when you are finished with it in order to prevent memory leaks, but this is no longer strictly necessary as they are added to a list of *LocoSourceEntry* Locos, which is accessible via *dccexProtocol.roster->getFirstLocalLoco()*. This can also be cleared with *clearLocalLocos()*, and it is also cleared along with the other lists when calling *clearAllLists()*.

This also means if you are creating a local roster in your software that you wish to be a part of the roster list, you must use the *LocoSource::LocoSourceRoster* type when creating the Loco object.